If the method name starts with self that indicates a singleton method—in normal usage that would be a class or module method. If the method name starts with self?, that makes it both a public singleton method and a private instance method.

If the method has multiple type signatures, you can separate them with a pipe character. Typically you'd do that on multiple lines:

```
def add_money: (Integer) -> Integer
             | (RomanNumeral) -> RomanNumeral
```

You can have an arbitrary amount of overloads.

The parameter list can be empty, in which case you do include the empty parentheses:

```
def price_in_cents: () -> Integer
```

Take a deep breath here while we go through all the ways in which to specify parameters…

A required positional parameter is denoted just by the type optionally followed by the name of the parameter:

```
def add(Integer)
```

```
def add(Integer other)
```

An optional positional parameter—meaning a parameter with a default value—is specified with the same syntax, but with a ? prefix to the type:

```
def expand(?String)
```

```
def expand(?String delimeter)
```

A splat parameter is denoted with a splat before the type name, with or without the name of the variable:

```
def join_all(*String)
```

Keyword parameter syntax is a little different. The keyword is followed by a colon, then by the type. Optional keywords—again, meaning keywords with defaults—are prefixed with a ?, and the type double-splats are preceded with ** (we'll show how to specify hash types in the next section):

```
def lots_of_args(arg1: String, ?arg2: Integer, **HashType)
```

A block argument has its own syntax:

{*parameters* ‹[self: *self_type*]› -> *return_type*}

?{*parameters* ‹[self: *self_type*]› -> *return_type*}

If the block is optional, the ? prefix is used. The parameters in the block have the same syntax as we just described for method parameters, and the return type can be any type:

```
def each_users_posts { User -> Array[Post] }
```

The optional "self type" is used when you expect the block to be a target of an instance_eval call and limits the type of objects that can call instance_eval on this block:

```
{ (User) [self: Document] -> Array[Post] }
```